

Lab Assignment: Python + Docker + PostgreSQL

Objectives

- Deploy a PostgreSQL database using Docker.
- Create a Python application connected to PostgreSQL.
- Perform database operations from Python (CRUD).
- Use Docker Compose to orchestrate multiple services.
- Manage data persistence with Docker volumes.

Context

A company wants to manage a product catalog.

You must develop a Python application that allows users to:

- Add products
- Display products
- Update products
- Delete products

The entire infrastructure must be deployed using Docker.

Expected architecture

The project must follow the structure below:

```
python_postgres_lab/
├── docker-compose.yml
├── db/
│   └── init.sql
└── app/
    ├── Dockerfile
    ├── requirements.txt
    └── main.py
```

PostgreSQL Deployment

Create a docker-compose.yml file.

Requirements

- PostgreSQL database
- Database name: inventory_db

- Username: admin
- Password: admin123
- Exposed port: 5432
- Persistent volume required

Questions

1. What is the purpose of a Docker volume?
2. Why use Docker Compose instead of multiple docker run commands?

Database initialization

Create an SQL script that runs automatically when PostgreSQL starts for the first time.

File:

db/init.sql

Create the following table:

Table: products

Column Type

id	SERIAL PRIMARY KEY
name	VARCHAR(100)
price	NUMERIC(10,2)
stock	INTEGER

Verification

Run:

```
SELECT * FROM products;
```

Question

Why is NUMERIC(10,2) preferred over FLOAT for storing prices?

Python application

Install the required dependency.

File:

requirements.txt

Required package:

psycopg2-binary

Database connection

Create a function:

```
connection()
```

that establishes a connection to PostgreSQL using:

```
host="db"  
database="inventory_db"  
user="admin"  
password="admin123"
```

CRUD Operations

A. Create a Product

Create the function:

```
add_product(name, price, stock)
```

Example:

```
add_product("Keyboard", 49.99, 10)
```

B. Read Products

Create the function:

```
display_products()
```

Expected output:

```
1 - Keyboard - $49.99 - Stock: 10  
2 - Mouse - $19.90 - Stock: 15
```

C. Update a Product

Create the function:

```
update_stock(product_id, new_stock)
```

Example:

```
update_stock(1, 25)
```

D. Delete a Product

Create the function:

```
delete_product(product_id)
```

Example:

```
delete_product(2)
```

Interactive menu

Create a menu allowing users to choose an action.

Example:

===== INVENTORY MANAGEMENT =====

1. Add a product
2. Display products
3. Update stock
4. Delete a product
5. Exit

Your choice:

Dockerizing the Python application

Create the file:

app/Dockerfile

The application must:

- Use Python 3.11
- Install dependencies automatically
- Start main.py when the container launches

Questions

1. What is the difference between a Docker image and a Docker container?
2. Why is it recommended to copy requirements.txt before the rest of the source code in a Dockerfile?

Execution

Run the application with:

docker compose up --build

Validation scenario

Add the following products:

Keyboard

49.99

10

Mouse

19.99

15

Display all products.

Expected result:

1 - Keyboard - \$49.99 - Stock: 10

2 - Mouse - \$19.99 - Stock: 15

Update:

Product ID: 1

Stock = 30

Then delete:

Product ID: 2

Expected final result:

1 - Keyboard - \$49.99 - Stock: 30

Reflection questions

1. What happens if the PostgreSQL container is removed but the volume is kept?
2. Why is the service name db used as the host in the Python application?
3. Which command displays all running containers?
4. Which command allows you to access the PostgreSQL container?
5. How can you view Docker container logs?

Bonus challenges

Level 1

Add a search feature:

search_product(name)

Level 2

Add a database constraint:

CHECK(stock >= 0)

Level 3

Create a second table:

categories

and associate each product with a category using a foreign key.